`batch_trials` development screen on `ic.koblitz_batched`

Local, diagnostic, counted-unit screen. Not a registered tournament round, not isolated, not promotable. Protocol `PROTOCOL.md` (frozen in `d98cc0d107`, Amendment 1 before any data in `60a451606b`). Instrument: `ic.koblitz_batched`, PR #3134 head `729847dcea` (the sessions record commit `39f147d96b`; `src/`, `Cargo.*` and `examples/` are identical between the two).

Result

Over round 0026's four development cells, the counted cost (group additions and doublings, GAE) of the tournament worker's pair-table pipeline **falls monotonically as `batch_trials` shrinks, down to 1**. Against the incumbent `batch_trials = 64`, `batch16` is cheaper by a factor of 0.690–0.799 (paired 95% intervals all below 1), and `batch1` by 0.643–0.771. Larger batches cost more: `batch256` is 1.80–2.27 times the incumbent. All 5120 measured records verified, and every measured record was replayed exactly (1280 replays per cell, all reproduced, 0 problems; the 320 warm-up records per cell are verified but not replayed), and the A/A control reads 1.000 on all four cells.

Two of the five predictions frozen in the protocol held (P2, P5); three did not (P1, P3, P4).

id	prediction (frozen before data)	outcome
P1	queries drawn are 8 for batch ≤ 8 and equal to the batch for batch ≥ 16	**not held**
P2	GAE rises monotonically from batch 16 to 256; 128 and 256 are the two most expensive arms	held
P3	GAE is non-increasing in batch from 1 to 8 (per-batch overhead)	**not held**
P4	the minimum over the nine arms lies in {4, 8, 16} on every cell	**not held**
P5	`ic-b16 / ic-b64` < 1, upper 95% bound below 1, on every cell	held

- **P1 not held.** Batches 32 and above draw exactly their batch (every run is one batch). Smaller batches draw more than eight queries on average, because a run needs a few relations beyond the 7 to 8 columns before the solve succeeds (table below). The three single jobs I pinned against the worker at batch 4 and 2 (n13a0, n17a1, n19a0) happened to need eight.
- **P3 and P4 not held, in the opposite direction.** There is no per-batch overhead in the counted unit: batch 1 is the cheapest arm on every cell (argmin: n13a0 b1, n17a1 b1, n19a0 b1, n23a0 b1), and cost rises with the batch all the way from 1.
- **P2 and P5 held.**

Paired ratio to the incumbent (`ic-b64`), counted GAE

Cells: `n13a0` ($\log_2$ r 10.97), `n17a1` (16.00), `n19a0` (17.00), `n23a0` (21.00); 32 public targets each, paired by target; cluster-bootstrap 95% intervals from `ecbench compare --save`. A ratio below 1 means the arm is cheaper than `batch_trials = 64`.

arm	n13a0	n17a1	n19a0	n23a0
`ic-b1`	0.643 [0.638, 0.647]	0.722 [0.718, 0.726]	0.741 [0.737, 0.744]	0.771 [0.769, 0.772]
`ic-b2`	0.647 [0.642, 0.651]	0.723 [0.719, 0.728]	0.742 [0.739, 0.745]	0.772 [0.771, 0.774]
`ic-b4`	0.651 [0.646, 0.655]	0.727 [0.722, 0.732]	0.745 [0.741, 0.749]	0.775 [0.774, 0.777]
`ic-b8`	0.660 [0.655, 0.665]	0.736 [0.730, 0.742]	0.753 [0.748, 0.758]	0.782 [0.780, 0.785]
`ic-b16`	0.690 [0.685, 0.694]	0.758 [0.753, 0.763]	0.773 [0.770, 0.777]	0.799 [0.798, 0.801]
`ic-b32`	0.788 [0.788, 0.788]	0.833 [0.833, 0.833]	0.846 [0.846, 0.846]	0.866 [0.866, 0.866]
`ic-b128`	1.425 [1.424, 1.425]	1.333 [1.333, 1.333]	1.309 [1.309, 1.309]	1.268 [1.267, 1.268]
`ic-b256`	2.274 [2.273, 2.274]	2.000 [2.000, 2.000]	1.926 [1.926, 1.926]	1.804 [1.804, 1.805]

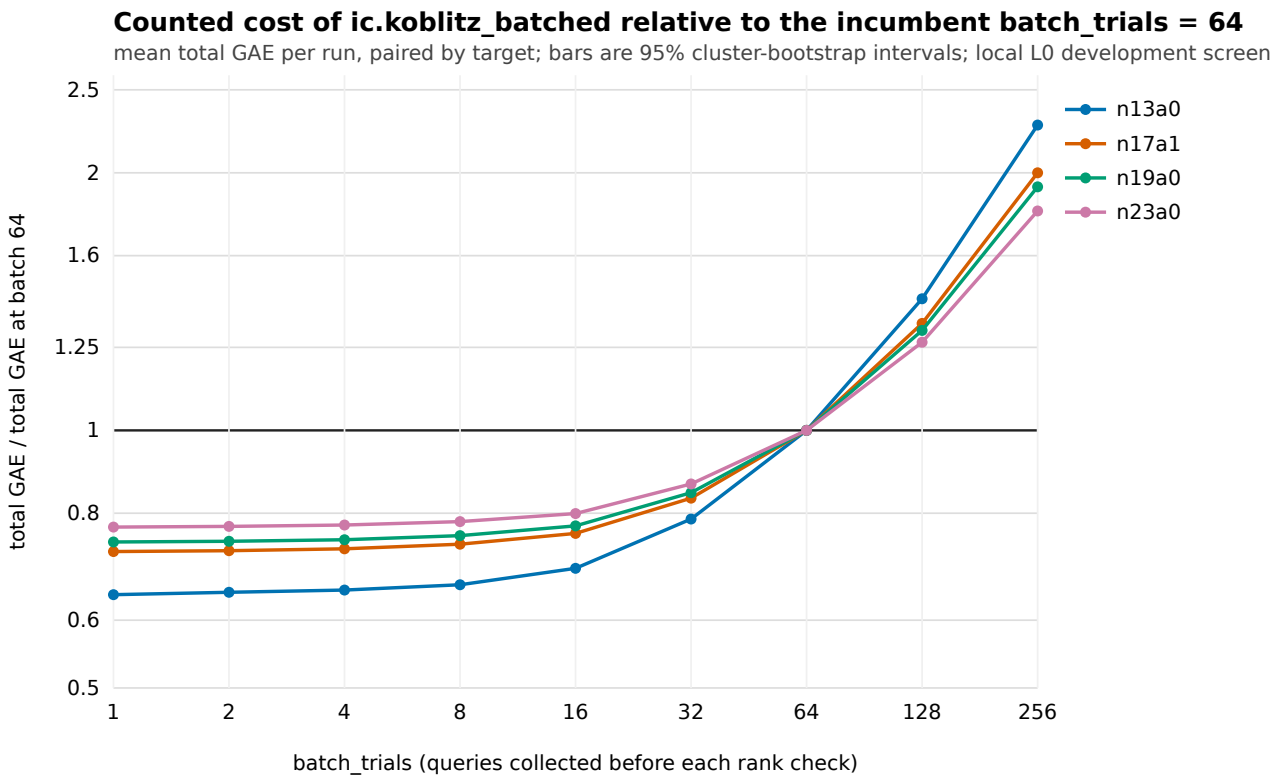


Figure 1: Counted GAE of each arm relative to the incumbent, by cell.

Why: the setup is most of the cost, and small batches stop sooner

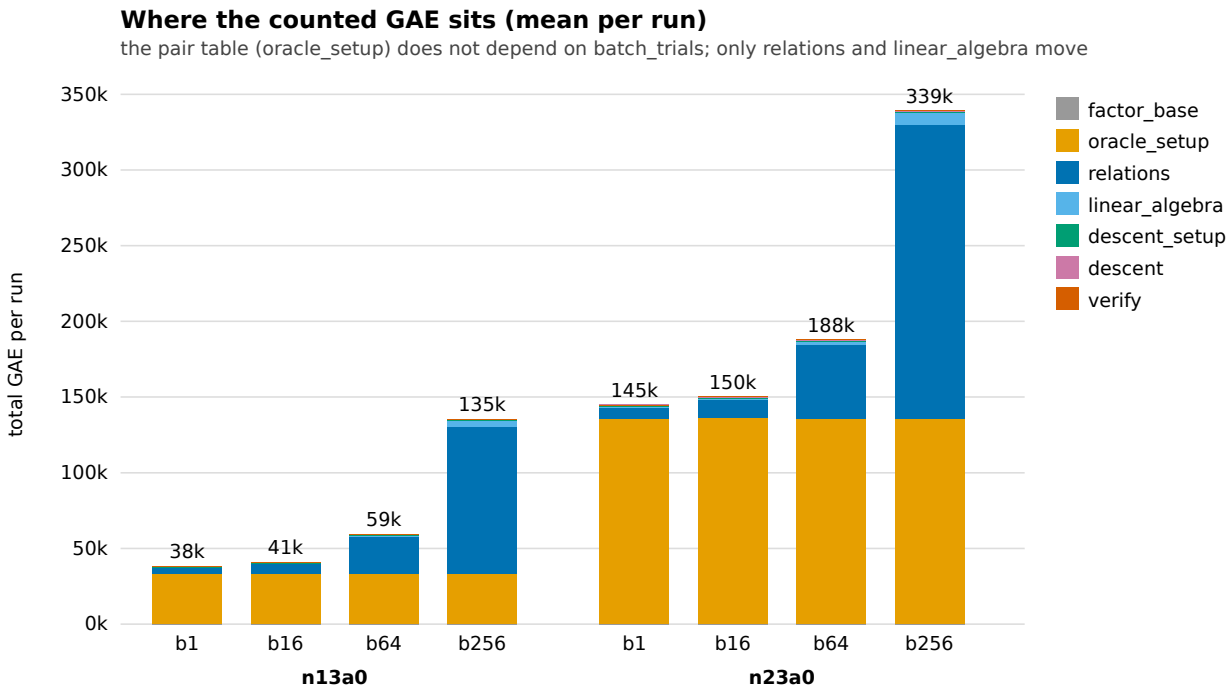


Figure 2: Counted GAE per phase (mean per run). `oracle_setup`, the pair table, is 56% (n13a0) to 72% (n23a0) of the incumbent's cost and does not depend on `batch_trials`; only `relations` and `linear_algebra` move.

At `batch_trials = 64` a run draws 64 queries and makes one solve attempt; at `batch_trials = 1` it stops after about 9.3–10.6 queries and makes 2.1–4.1 solve attempts. Each query costs a roughly constant number of GAE in the relation phase, so the surplus queries of a large batch are the whole difference. Mean queries drawn / mean solve attempts per run:

arm	n13a0	n17a1	n19a0	n23a0
b1	10.1 / 4.11	10.6 / 3.59	10.2 / 3.20	9.3 / 2.14

arm	n13a0	n17a1	n19a0	n23a0
b2	10.8 / 2.38	10.9 / 2.44	10.5 / 2.26	9.7 / 1.66
b4	11.3 / 1.84	11.6 / 1.90	11.1 / 1.78	10.4 / 1.41
b8	12.8 / 1.59	13.3 / 1.66	12.8 / 1.60	12.0 / 1.31
b16	17.2 / 1.08	17.5 / 1.09	17.0 / 1.06	16.1 / 1.01
b32	32.0 / 1.00	32.0 / 1.00	32.0 / 1.00	32.0 / 1.00
b64	64.0 / 1.00	64.0 / 1.00	64.0 / 1.00	64.0 / 1.00
b128	128.0 / 1.00	128.0 / 1.00	128.0 / 1.00	128.0 / 1.00
b256	256.0 / 1.00	256.0 / 1.00	256.0 / 1.00	256.0 / 1.00

The extra solve attempts of the small batches are cheap in the counted unit, and also in the native work the unit leaves unpriced: the sparse-solve row operations per run fall from 1530 at `batch64` to 109 at `batch1` on n13a0 (n19a0: 2461 to 110), and the pair-table bucket reads and filter probes fall with the batch too. In the five unpriced native counters recorded (sparse-solve row operations, pair-filter probes, bucket reads, Frobenius maps, canonicalisations), `batch16` is not above `batch64` on any cell: yes.

What this does and does not show

- **Direction agrees with round 0026.** Round 0026 measured `batch16` at 0.630× the incumbent in Valgrind instruction reads. Here it is 0.69–0.80× in GAE. The units differ and the magnitudes are not comparable; the sign agrees.
- **The counted unit cannot find the true minimum.** GAE prices group operations only. It does not price the fixed cost of one solve attempt (building the matrix, allocation), field arithmetic outside the group law, hashing, or any wall time. Those costs are what push back against very small batches, and the measurements that see them (instruction reads, isolated wall time) are the ones round 0026 and the open isolated confirmation use. So this screen does **not** show that `batch_trials = 1` is best; it shows that, on the work GAE and the recorded native counters see, nothing counts against small batches.
- **Round 0026's panel stopped at 16.** Of its default candidates, `batch16`, `batch32` and `batch128` varied `batch_trials`; the tournament's own refinement rule around a selected batch (`batch/2`, `batch/4`, `2·batch`, `4·batch`, 1, 2, 8, 256 in `src/bin/icprog/tournament.rs`) would try 8, 4, 2 and 1 next. This screen says those are worth including in the next registered, isolated round.
- **Wall time is not measured here.** The sessions are isolation L0 (`--cpus none`); the wall figures in the records are descriptive only. The open round 0026 question, wall time under a core reservation, needs the isolated round.
- **Not claimed:** anything about larger curves, other factor bases, `dense`, `excess8`, `window8`, rho, or scaling. No cross-cell pooling. Nothing here is promotable.

Evidence and reproduction

- Executed by `tools/task_card.py run` on `CARD-ecbench-batched-screen-20261101-1` (63 commands, exit 0, no retries); the director's assessment (`cards/card-1.assessment.md`) is `executed_as_specified`.
- `audits/<cell>.verify.txt`: `ecbench verify --replay-all` on each session, 1600 records verified and 1280 replays reproduced, 0 problems. The ecbench binary is `447a6d3c4842…` in all four sessions.
- `summary.json` is made by `figures/make_summary.py` from the verified records and the saved comparisons; this report is made from it by `figures/make_report.py`. `comparisons/` holds each arm's comparison text.
- The four `sessions/<cell>/records.jsonl` files (17 MB each) are in the artifact store, with an `.artifacts.json` manifest per cell; `python3 tools/artifacts/artifacts.py pull research/ecbench_batched_screen_20261101/sessions` hydrates them. Run on them, `figures/make_summary.py` reproduces `figures/summary.json` byte for byte.